Computer Graphics: Lecture 5

Buffers and Colors

Welcome Back!

Last time we drew a triangle!

It was kind of an adventure getting it to draw.

But now that we've done that, we can just scale it up to drawing complex 3D models, right?

Not so fast

Not quite. The way we did it (putting all the vertices in the shader) is not a way that will scale.

Instead, we want to store the vertices in an array in GPU memory, called a buffer. Then we want the vertex shader to read out of that.

Additionally, we want to add some color. The fragment shader is just drawing a solid color, which isn't very interesting.

The Gameplan

First, we will modify our sample as follows:

- Change the shader to receive data from a buffer
- Create a buffer which stores the triangle data
- Tell the pipeline to get its data from a buffer
- Bind that buffer with a command so that it's there when needed.

Then we will learn how color works in detail. There is surprisingly a lot more to it than just RGB.

Finally, we will add some color to our mesh and have it smoothely blend between colors. That will be your next project!

Vertex buffers

The main change we're going to make is to learn to use a vertex buffer.

A vertex buffer stores all the information each vertex needs.

What information is needed by a vertex? It's just a point in space…

Actually, we store tons of things in a vertex:

- Its position
- Its color
- Its texture coordinates (i.e., where on a detailed surface image this vertex's color comes from). This might replace the color info.
- Its normal (a vector pointing exactly away from the mesh)
- Potentially multiple texture coordinates, animation bones, etc.

Vertex buffers (2)

The GPU is already set up to read from a vertex buffer, we just have to tell it that there will be one, and then make it available.

This means we need a somewhat more complicated pipeline.

It also means we need to create the buffer, and make sure we issue a command to load it.

The vertex shader will then receive its parameters from the buffer instead of using `vertex_index` to look them up in an array.

Changing the shader

This is a shader that pulls its information from a vertex buffer:

```
@vertex fn vs(@location(0) vertPos: vec3f)
-> @builtin(position) vec4f
{
    return vec4f(vertPos, 1.0); // expand the 3D coord to 4D
}

@fragment fn fs() -> @location(0) vec4f {
    return vec4f(0.4, 0.8, 0.3, 1.0);
}
```

What changed?

Previously, the input was `@builtin(vertex_index) index: u32`

Now, we write `@location(0) vertPos: vec3f`

When we use `@location(...)` in a parameter to the vertex shader, that argument will be extracted from a vertex buffer. In this case, from the 0th **attribute** (each piece of information in the vertex buffer is called an **attribute**)

Note: the location in the fragment shader is *completely different*. The `@location(0)` it returns means that the result is written to the 0th attachment. It is a completely separate reference than the input to the vertex shader. It has nothing to do with vertex buffers.

Now what?

Now, let's also create the data for a vertex buffer. We're going to use a special built-in Javascript type called `Float32Array`:

```
const vertData = new Float32Array([
    -.75, -.75, 0, // first vertex
     .75, -.75, 0, // second vertex
       0,  .75, 0, // third vertex
]);
```

This is a Javascript array where the elements are densely packed next to each other in memory (like an array in C, or a numpy array in Python).

Note: this *isn't* on the GPU yet. This is a normal array in RAM.

Creating a buffer on the GPU

Now we need to create a buffer on the GPU. The WebGPU driver will allocate the memory for us, but we need to tell it how much.

```
const vertBuf = device.createBuffer({
    size: vertData.byteLength,
    usage: GPUBufferUsage.VERTEX,
    label: "triangle vertex buffer",
    mappedAtCreation: true,
});
```

This creates a new buffer. Let's go over each of its fields…

Buffer fields

First was `size`. This is the size in bytes. We have 9 floats, and each float is 4 bytes each, so we expect the buffer to be able to fit 36 bytes of data.

Then there's `usage`. This tells us how the buffer will be used. This is important because video drivers often store things differently in memory depending on how it will be used. In our case, we announce our intention to use this buffer as a vertex buffer.

The `label` is there for debugging. If we use our buffer incorrectly, it can tell us *which* buffer was problematic.

Finally, theres `mappedAtCreation`...

Memory Mapping

The GPU can have its own RAM separate from the CPU's RAM.[1]

One of the fastest ways to write data to separate areas of RAM is to use a feature called **memory mapping**.

Memory mapping uses the memory controller on your CPU to make it so that when you write to a particular address, the bytes end up somewhere else.

`mappedAtCreation` means that WebGPU will map the buffer.

[1]This isn't strictly required. In some systems the GPU shares memory with the CPU. This is called a "unified memory architecture". However, WebGPU is not built to assume that you have a unified memory architecture, because many systems don't. (integrated GPU systems usually do, discrete GPU systems usually don't)

Using the mapped buffer

The typed arrays, such as `Float32Array` have the ability to point to any memory area you choose.

You can, e.g., have a buffer with 10 elements, and have two different `Float32Arrays` pointing into it. They can point to the same elements or be offset.

To get the memory mapped address of the buffer we created, we call a method called `getMappedRange`.

We create a `Float32Array` that points into that range, and then copy the floats from our local data into the GPU…

Using the mapped buffer example

```
// this guy is now pointing at some memory that will
// be copied into the GPU and overwrite the buffer.
const mapped = new Float32Array(vertBuf.getMappedRange());
mapped.set(vertData); // .set(...) performs this copy
```

Since we're only using the second `Float32Array` once to copy some data over, we can combine creating it and setting its data like this:

```
(new Float32Array(vertBuf.getMappedRange())).set(vertData);
```

When done, it's very important to unmap it:

```
vertBuf.unmap();
```

Questions?

Using the mapped buffer example

Now we have a buffer, but our pipeline needs to know that a buffer will exist when we draw next time. The changed part is the `vertex` field:

```
vertex: {
  module: shaderMod,
  buffers: [{
      attributes: [{
          format: "float32x3",
          offset: 0,
          shaderLocation: 0,
      }],
      arrayStride: 3 * 4,
  }]
},
```

Explaining the fields

Every piece of information we associate with a vertex in the vertex data is called a **vertex attribute**.

Right now we have 1 attribute: position.

Soon we will have a 2nd one: color

Each attribute has a location. We coded our shader to assume that `@location(0)` was the vertex's position, so we use `shaderLocation: 0`

The format is the datatype. Here, `float32x3` means a vector of 3 floats.

Offset is the number of bytes into the buffer of the first vector of that attribute. Ours has no data before it, so use `0`.

What is “stride”?

Stride is a bit confusing.

It’s the number of bytes *between* two instances of the same attribute.

So its the number of bytes between each `position`.

Since each position is 3 floats, and each float is 4 bytes, there are 12 bytes between each position.

The GPU will use the offset to find the attribute of the first vertex, then multiply the stride by the vertex index. This regular storage layout allows the GPU to process vertex it needs at any time in constant time without having to search through a linked list or something.

Using the buffer?

So, we made a buffer and loaded it with data. Our pipeline is expecting there to be a buffer. How do we actually use it?

We have to **bind** the buffer. Meaning, we tell WebGPU that there is a buffer at buffer location `0`.

Why zero? Because in our pipeline description, we only have one buffer, and it's the zeroth element in our array. We could have many buffers, one for each attribute, and we would then need to bind them all.

```
// from the pipeline description
buffers: [{ // <- notice that `buffers` is an array of objects.
  // ... there could have been more than one buffer.
}]
```

Using the buffer (2)

But for us, we told our pipeline to expect 1 buffer with the 1 attribute:

```
pass.setVertexBuffer(0, vertBuf);
```

We're saying: "set the zeroth buffer to use `vertBuf`"

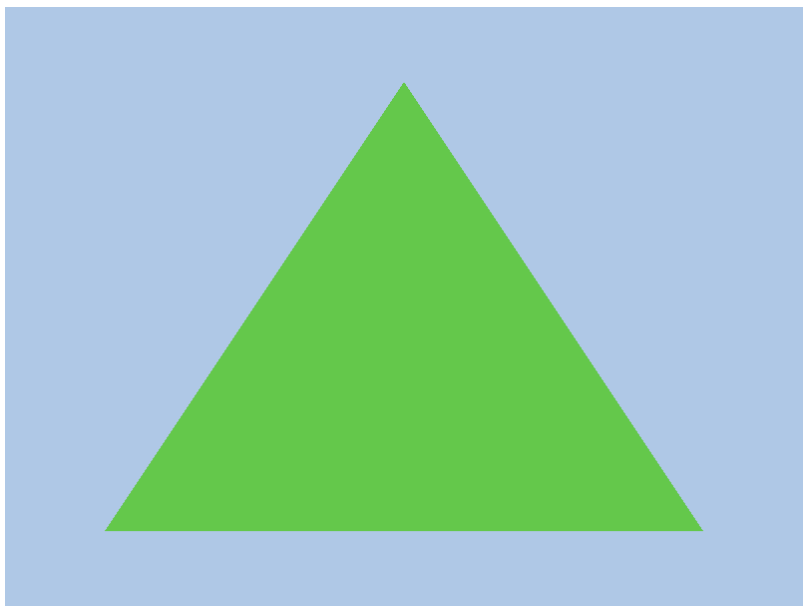
Note, we use the buffer on the GPU that we created with `device.createBuffer(...)`. We don't use a `Float32Array`.

The buffer object is basically a pointer or handle to some GPU memory.

Then we draw like normal:

```
pass.draw(3); // everything else is the same
```

What do you see?



Hopefully the same thing. If not, let's debug!

Questions?

Adding colors

Now it's time to understand colors better.

What is there to understand? Quite a lot actually.

Did you know that most people can see a lot more colors than your screen can show?

Or that some screens actually have wider ranges of colors they can display, and that you can actually choose which color profile you use when creating your canvas context?

Read on!

Why do we use RGB?

We've been using RGB (red/green/blue) to describe colors.

Why don't we use some other colors as our basis, like "gray, periwinkle, burnt-umber" or something?

[What's so special about RGB?]

Light

It has to do with how our eye responds to light.

What even is light?

It's electromagnetic radiation in a particular range of wavelengths that a typical eye can respond to.

The kind of light we're interested in in this course is the kind most humans perceive.

How eyes typically respond to light

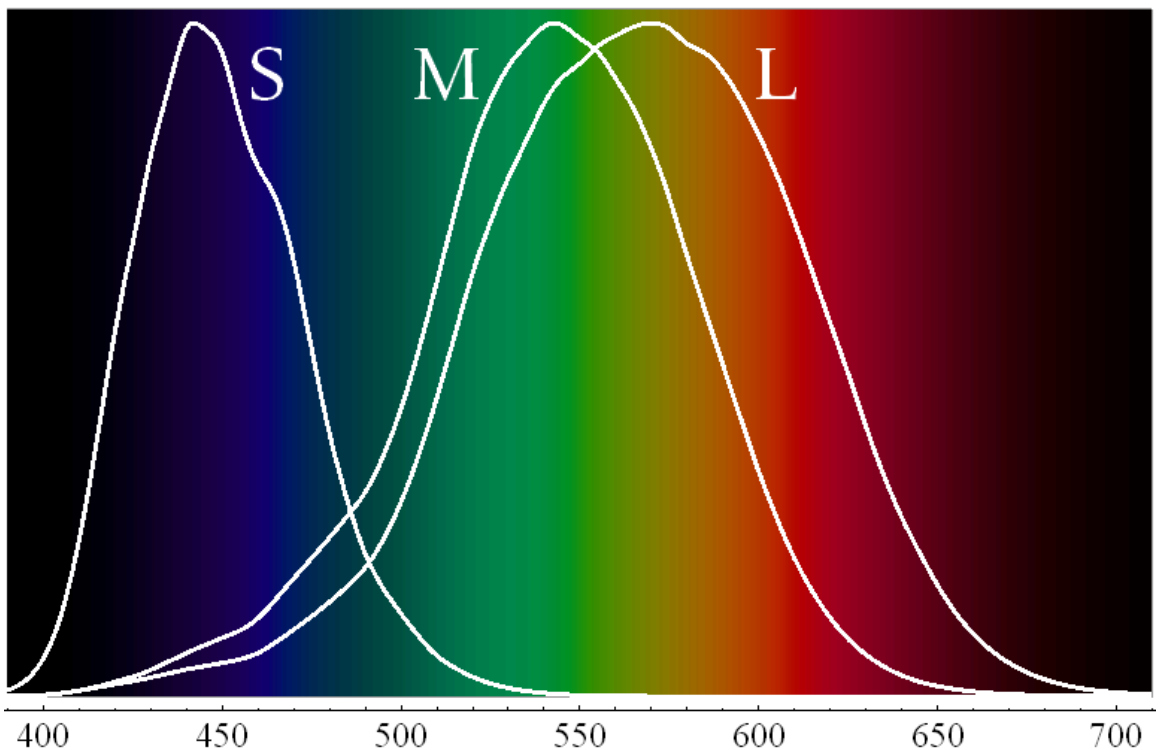


Image source, generated from cone sensitivity data from here.

How eyes typically respond to light (2)

A typical human eye has many instances of three types of color cones spread throughout the surface of the retina.

The three types of cones are named after the wavelength of light they react most strongly to. **S** cones react to short wavelengths (blues and purples), **M** cones react mainly to green, and **L** cones mainly to yellow.

It's fuzzy though. You might have thought that one kind of cone *only* reacted to red, and another *only* to green. They bleed into each other, and not everyone's eyes react the same.

How eyes typically respond to light (3)

Some sighted people do not experience as much response from one or more of the kinds of cones.

These condition is called *colorblindness*. When making graphical software, be aware that some people experience colors differently.[1]

Rarely, some people may possess 4 cones. They are called Tetrachromats, althought this seems to have only been demonstrated once under scientific conditions. The fourth cone's response function was between the M and L cones.

[1] I've tried to ensure this course and the assignments are colorblind accessible. If you have colorblindness and believe it will hinder you from completing an assignment, please let me and the access center know so that we can modify the course.

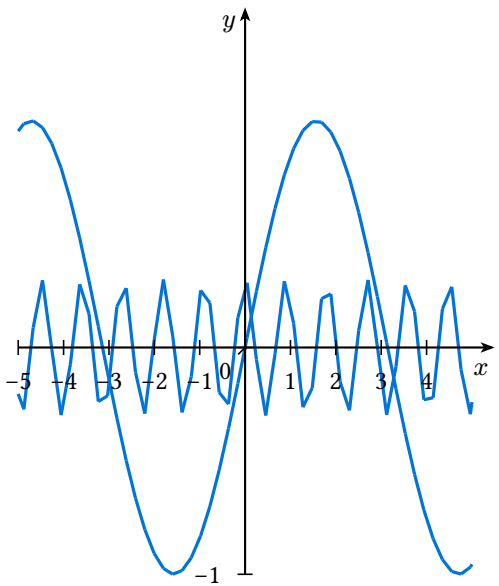
This raises a question

Here's the first question that might pop up after looking at that graph:

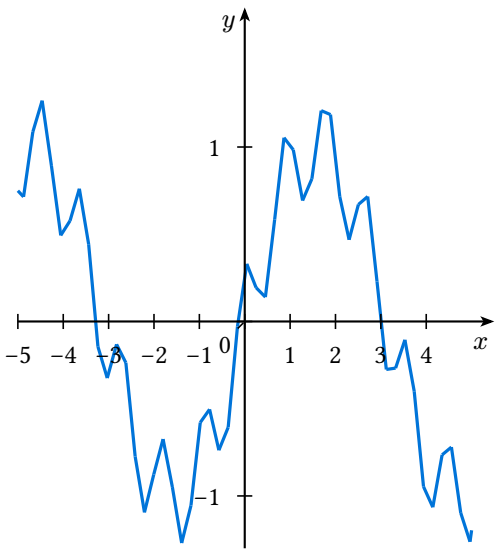
"Why isn't color a one-dimensional value"?

After all, if the color is one dimensional, and our cones react to different wavelengths, why not just store the color itself? Why store how much each wavelength is present?

Because lights are added together



Before adding two waves



After adding the waves.

This light might stimulate 2 different cones at once, and might look ‘pink’, which isn’t a color on the line.

Constructing colors

The light spectrum is one dimensional, it shows how color changes with increasing frequency of the electromagnetic signal.

Different parts of the spectrum excite different cones.

However, there is no reason why we have to look at one wave at a time. We could shine two different color lights, and their waves will add.

This can stimulate combinations of cones that a single sinusoid could not stimulate. For example: L and S at the same time.

Constructing colors (2)

Also: there may be more than one way to generate the same color.

Yellow light can be generated with a sine wave at ~520 THz.

It can also be generated by adding pure red and pure green light.

The resulting waves can be perceived the same by someone with typically functioning eyes, but will have completely different mathematical functions.[1]

This is called *metamerism*.

[1] in fact, they will also reflect differently on non-white surfaces. Also, according to the analysis of Sharp Quatro displays I cite later in the presentation, many people can distinguish natural yellow from monochrome yellow.

Constructing colors (3)

So, we use red, green, and blue as the bases of our color vectors because they are the most independent colors which excite specific cones.

We can also cover the entire spectrum with combinations of red, green, and blue.

However, most people can perceive colors that are not on the color spectrum, such as pink.

Question for discussion: [Do you think red, green, and blue are enough to generate every color you can perceive? Why or why not?]

Questions?

Surprisingly no

You might think that the existence of “photorealism” means that we must be able to represent every color, but even photos have trouble capturing them all perfectly.

There are some colors that you can see that your monitor cannot display. And not all monitors are equal either.

Different monitors support different color profiles, which describe its **color space**.

Color spaces

If we think of red, green, and blue as separate dimensions, we could imagine a 3D volume that contained all the colors that are displayable.

This 3D volume does not have to be a cube. There is no reason the color response has to be linear.

However, what is the color space of the human eye?

It turns out, it's also non-linear.

In fact, how do we even know what colors are typically visible?

CIE experiments

The International Commission on Illumination (abbreviated CIE for its French name: Commission internationale de l'éclairage) was formed as an international body to standardize color specifications from prior experiments done on color perception.

The experiments worked like this:

- Shine a light onto a screen in front of an observer.
- Next to the screen was another light that the observer could control.
- The observer could adjust the power of red, green, and blue lights for their light to try to match the other light (i.e., the target light)
- The experimenters would record the combination.

The problem

However, as I said before, some colors cannot be made as combinations of just red, green and blue.

In these cases, situations would arise where the controlled color could not be matched under any circumstance.

In these cases, the researchers would try to simulate “negative colors”. They would add one of the primary colors to the target until it was “in the range” that could be matched.

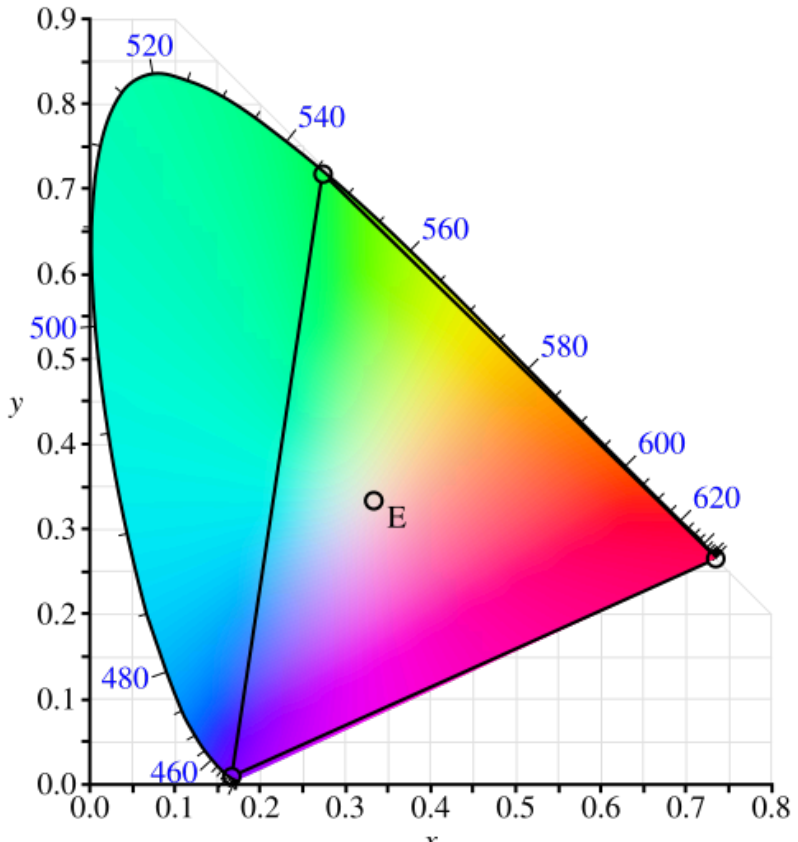
The amount of color they had to add to the target was recorded as “negative color” of the given primary color.

Showing a gamut

If we take a 3D volume of all the colors humans can see, and we take a cross section (i.e., for constant brightness), we end up with a shape called a **color gamut**.[1]

Here is CIE's RGB space plotted over the whole visible gamut.

Image by BenRG



[1] "Gamut" just means "range". It often found in the set phrase "run the gamut".

The RGB space

That RGB triangle was the space that could be represented using combinations of red, green, and blue.

The curvy region outside of it is the region that required “negative colors”.

Comprehension check: [why is the minty blue in that region so large and indistinct]?

Comprehension check answer

Because the projector/your monitor can't display those colors!

That diagram shows how much perceivable color we give up when we insist on using RGB.

How many colors would we need to represent all of it?

It depends on how you think about it.

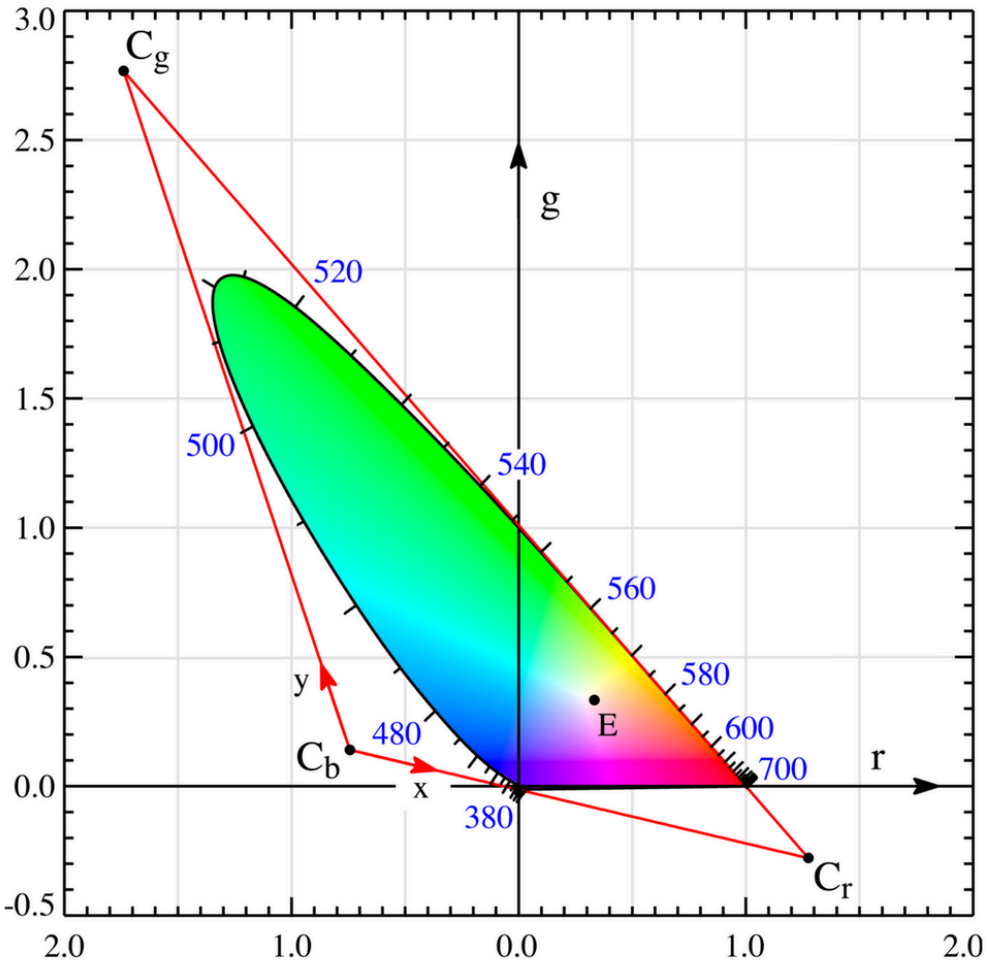
- We could add new colors to create a complex convex shape that included the whole gamut without including any non-visible colors…
- Or, alternatively, we could keep the triangle, but make the bases extreme enough that it could cover the whole gamut (including colors that we can't see)

The XYZ space

The latter exists, and it's called the CIE XYZ space.

X, Y, and Z aren't colors. They are arbitrary bases from which the actual color can be reconstructed.

The idea is that a combination of 3 values can represent any visibly perceivable color if you make the triangle big enough.



Public domain image

Why no monitor uses the XYZ space

Unfortunately, those bases aren't real. There is no "x" colored light. They are a just way to "name" colors.

Also, because these values have to represent any visibly perceivable color in a triangle, there are also non-perceivable colors inside it, which would not be ideal.

So there are no displays that support "XYZ" color.

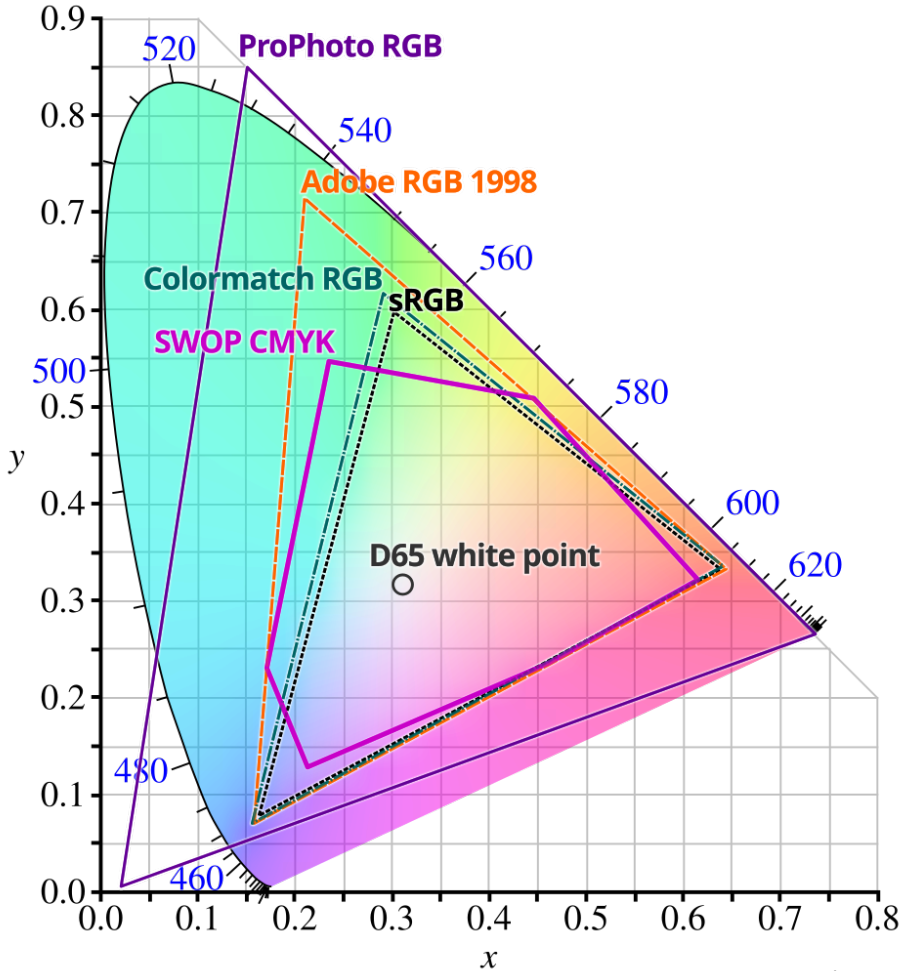
Actual color spaces

So you might think “darn, we have to go with RGB then. So we’re giving up all those colors.”

It’s actually worse than that. You *wish* your display supported CIE RGB.

Actually, most displays are (currently) using the sRGB space.

Look at that tiny triangle…



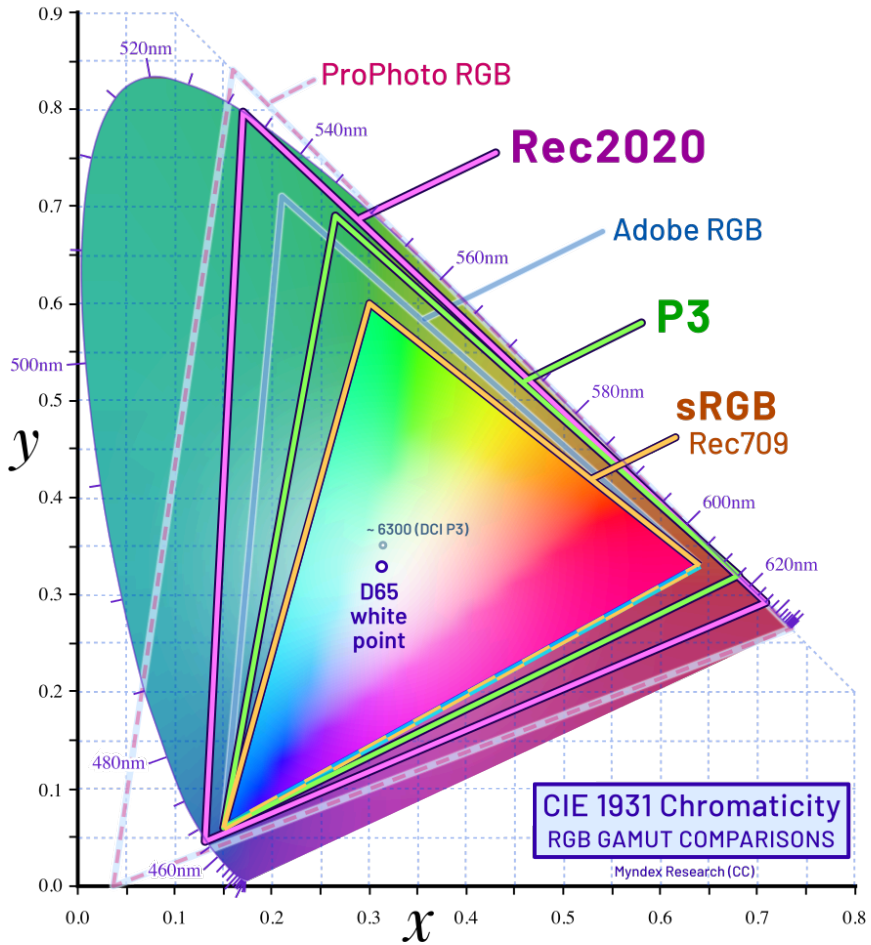
You might have a better one

Some monitors support the P3 colorspace.

You can test to see if yours does.

This is a choice you make when creating or canvas. It will change how colors look, so you need to be very mindful of the choice.

I will use sRGB for this class for compatibility.



Questions?

How do the displays work?

There are actually numerous technologies. The most popular are/were:

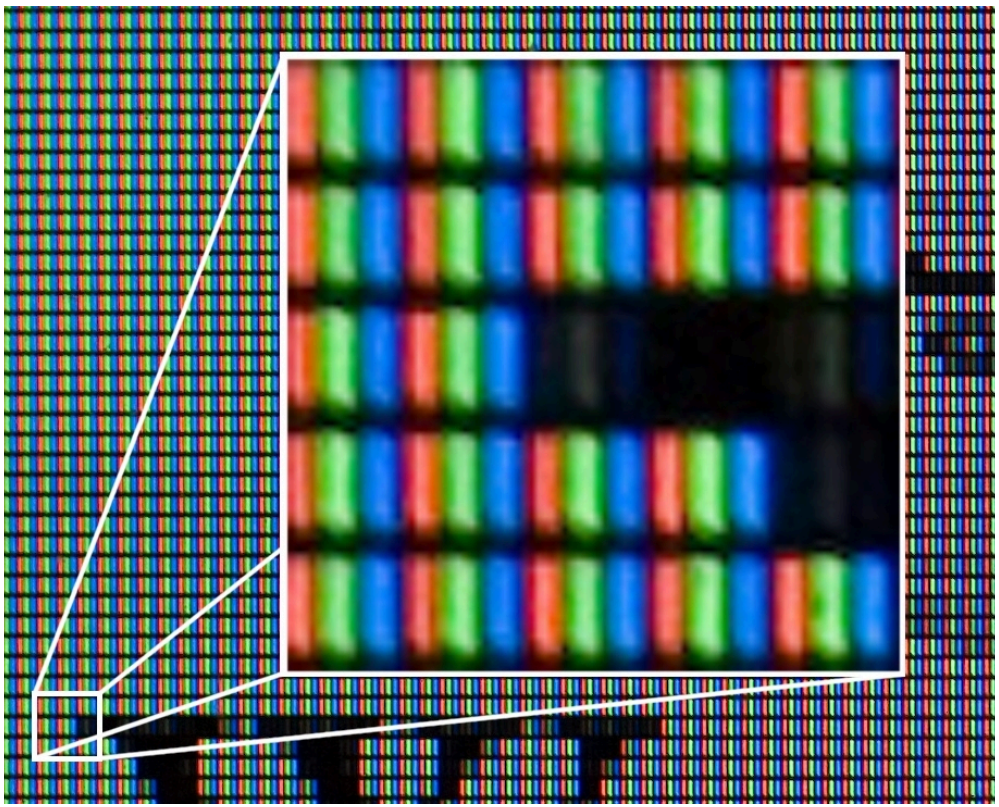
- Cathode ray tubes (CRTs) are the oldest. They project a stream of electrons at a phosphor-coated screen. The specific voltage function of the electrons would cause different phosphors to light up for colors.
- Liquid cristal displays (LCDs) have separate backlights that emit the most intense red, green, and blue. Liquid crystal cells, when powered, change their structure to become less transparent, covering up some of the pure colors to different degrees.
- Organic light emitting diode (OLED) displays have red, green, and blue LEDs for each pixel.

Subpixels

Most consumer displays are either LCDs or OLEDs. Both of these have the concept of “sub-pixels”.

The sub-pixels are the portions of a pixel that generate a primary color.

Some text rendering algorithms target these for extremely sharp text (sup-pixel anti-aliasing)

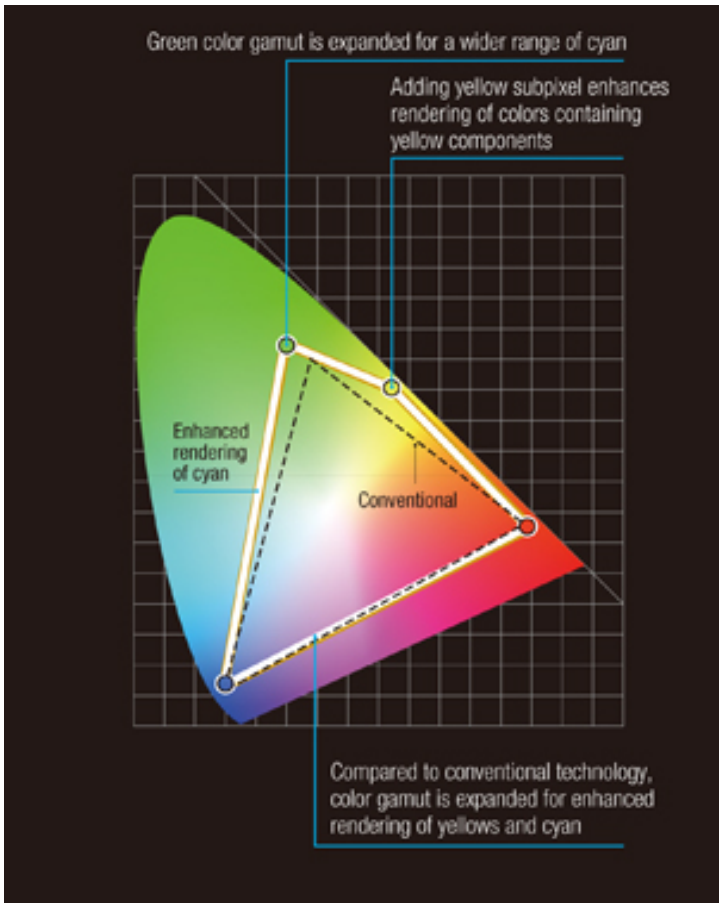


A zoomed-in picture of an LCD display. By User:Diliff, User:Ravedave - User:Diliff, CC BY 2.5

Funny historical note

Some of you might remember the Sharp Quattron

It was a TV that would supposedly have a wider, 4-sided color gamut by adding a “yellow” primary subpixel. (Sharp corporate press release)



Funny historical note (2)

Yellow is a strange choice though, isn't it? It barely moves the gamut, but at great expense.

Also, movies would have to be encoded with yellow pixel data, and none were. So what was it doing?

According to this analysis from Queen Mary University of London, there is a yellow sub-pixel, but it just lets in an amount of red and green light that could have been compensated for by making the red and green subpixels bigger. There isn't a separate monochrome yellow backlight.

So it seems like a waste of money…

Questions?

Is that it?

Okay, we've got a big grid of pixels, we understand those.

They're made up of subpixels, red, green, blue. We increase the intensities of those to make the colors we want.

Boom. Easy. In fact, I bet many of you knew all about RGB from just pop-culture osmosis, web work, or image editing.

But there's an important concept that is *not* widely discussed, and we need to cover it.

Gamma correction

Human perception of light is not linear.

A small increase in light intensity for a very dim level of brightness is perceived as a very large increase.

However, if the brightness is already high, the same linear increase will be barely noticeable.

It’s probably easier if I just show you:

Perceptually uniform brightness

This is what a perceptually uniform color gradient looks like.

It looks like the color is getting brighter at a constant rate.

I'm increasing the brightness by 1% per "slice".

Here's the thing: it's secretly being run through a function. If we passed raw brightness values (i.e., referring to the voltage on an OLED or something) it would not look like that.

Uncorrected brightness

Here's what it would look like if we did that:

This is a color ramp with linear light. Notice how much faster it gets bright, and how it's kind of "blown out". At like 20% brightness it's already at 50% perceived brightness.

This is the one that "seems" uniform but is actually being corrected.

Corrected how?

Gamma Correction

The sRGB standard also tells that input colors should be assumed gamma corrected. Other standards are similar.

This is the correction: $\text{Brightness}_{\text{out}} = (\text{Brightness}_{\text{in}})^{\gamma}$

In sRGB, a good approximation of γ is 2.2. We're raising the input color signal to a power. Technically γ changes: it's piecewise, but 2.2 is close.

Remember that the color brightnesses are between 0 and 1 normally. So this is going to make the color *dimmer*. It will not get bright as fast.

This transformation is applied to the 3 color channels independently.

Gamma encoding

Let's say we're storing an image and we write a brightness value of 10%

If our image is saved as sRGB, that means we mean 10% *perceived*.

If we are reading 10% from an actual camera sensor or something, we need to apply the inverse encoding: raise $10\%^{0.45} \approx 35\%$[1]

That is, 10% of actual measured brightness will be a lot brighter than you'd think, so we represent it as 35% preceived brightness.

[1] .45 is just 1/2.2

We usually assume that pixels are post-correction

If you load an image file, the pixel values in it are just numbers. We don't know what gamut or gamma unless it's marked.

Almost all images are going to be sRGB.

WebGPU let's you describe the format yourself. When you load a texture, you can tell it whether it's corrected or not. We'll discuss how when we get to textures.

Why do we care?

Ordinarily, this transformation is very useful. You probably didn't know about it, but it made the RGB percentages "make more sense" and match the typical person's visual perception.

Unfortunately, when we do blending with light, for example, adding light compositions together, we want the actual raw light values. *Not* the perceived light values.

So to make the calculations work out, we need uncorrected values.

And our textures are going to be corrected, so we need to know that so we can sample the linear colors (not the gamma corrected ones).

That's all for now

This lecture covered two very different things:

1. Adding buffers to our pipeline
2. Color perception

It's a lot to cover. Make sure you can get the sample to work, and try to build it from scratch. Ideally, start from the encoder and work backward, following error messages (you will probably have to peek at at the WGSL though, but do your best ot learn it).

I recommend working through the book chapter, too, as well as reading this excellent article on gamma correction. This article has numerous examples of incorrect gamma correction causing wrong images.

Next time

We're going to actually use what we've learned to blend some colors. And do it in a gamma-correct way!

We're going to add colors to our buffers.

We're going to make a multi-colored triangle with smooth blending.

We're going to learn about interpolation and "stride".

Questions?